

01.数据库

数据库相关的问题出现频率是第一高的，几乎是必问，必须掌握，并且面试官基本对Mysql的底层都非常熟悉，所以这里没办法糊弄过去（指单纯背背概念就能过关）

我这里仅列出对同学们有难度并且频率高的

1. Mysql索引相关问题（难度：★★★）

索引类型

！ 首先强调mysql中索引是有好几种类型的，

结构上来说主要有 B+Tree 默认索引、 Fulltext 全文索引、 Hash 哈希、 R-Tree 空间索引等

功能上来说有主键索引、唯一索引、普通索引、复合索引

我们平时用的是引擎InnoDB默认都是 B+Tree

索引作用

先理解索引的作用再去记忆，不要小看这一步

！ 数据库我们可以先简单理解成一条条记录存储在文件里

正常情况下如果要查询某个数据我们需要挨个遍历这些所有数据，自然查询起来就会很慢而索引就是一个**有序的数据结构**（通常是 B+Tree），因为有序所以查询起来就快很多比如字典里的目录其实就是根据字的首字母来排序

但是查询虽然快了，每次增删改不仅要维护数据，还要去维护这个索引，所以**增删改的性能会受到一定影响**

B+Tree结构

！ 先记下图

特点总结：树本身有序，所以搜索快

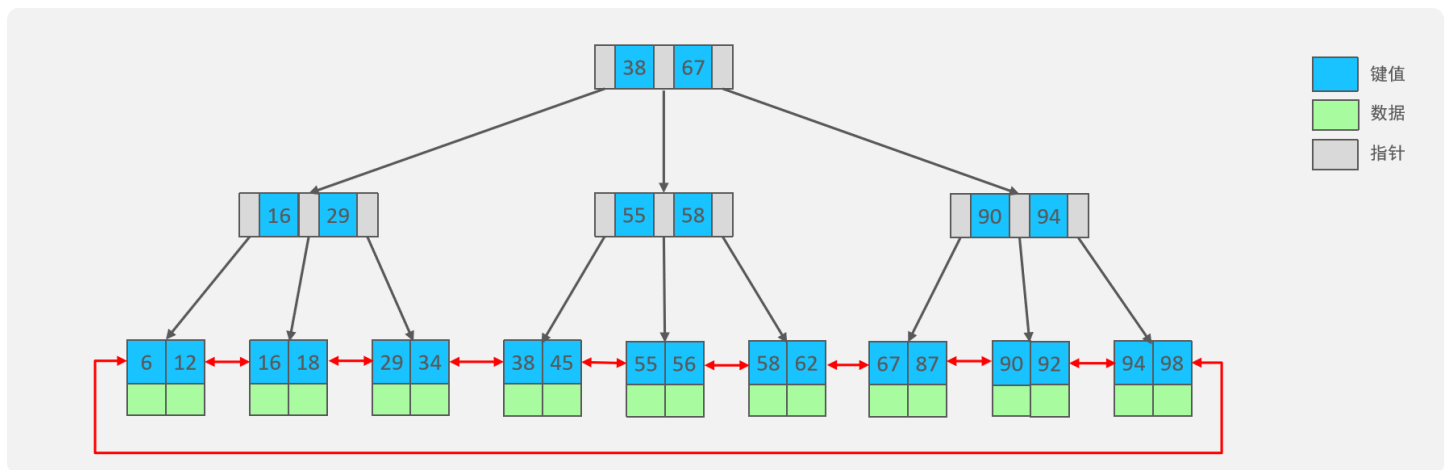
数据存储在叶子节点，所以每次查询都必须查到叶子节点才结束，所以**每次查询的效率不会变化很大**

并且叶子节点上形成了个**双向链表**，范围查询更快

每个节点里会存储**多个key**，并且有**多个子节点**，所以**层高更低**，层高低就**IO少**，所以效率更快

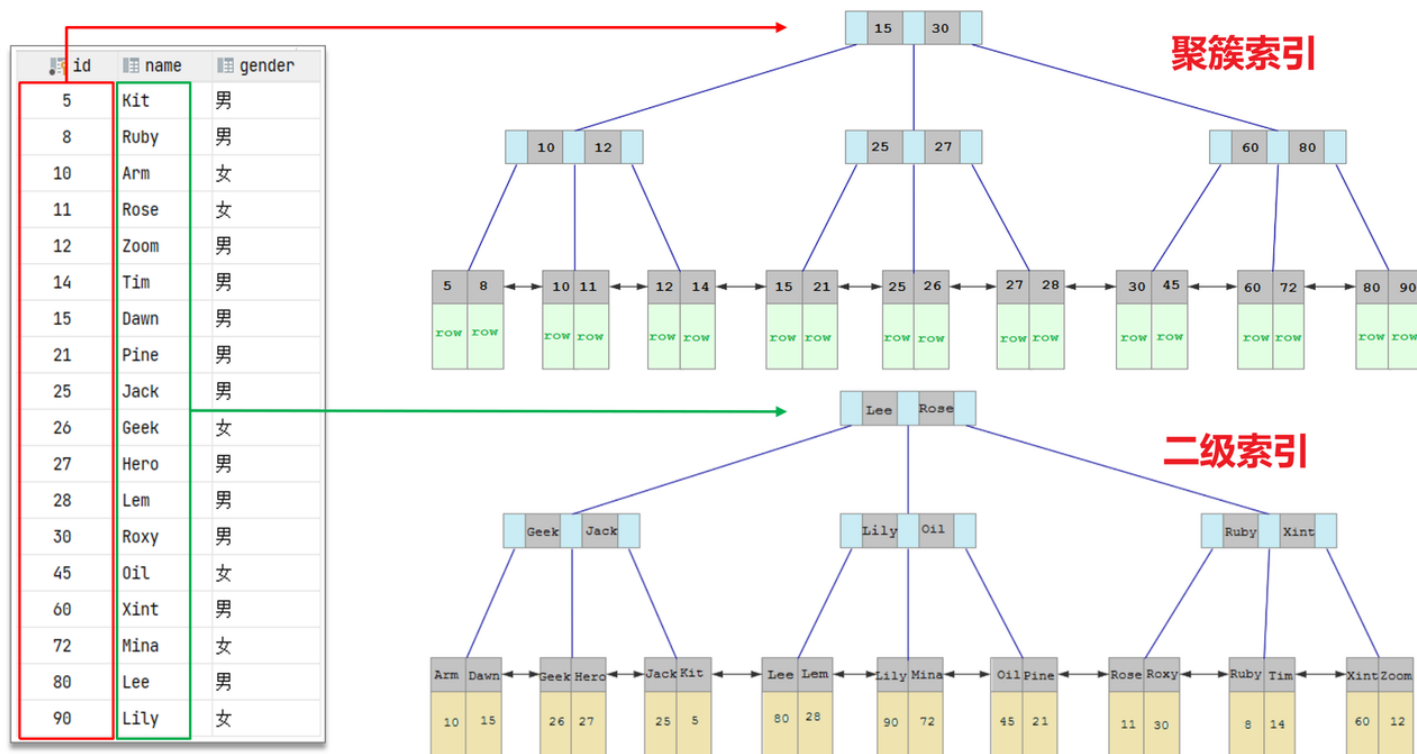
相关面试题：**B树和B+树的区别**

B树非叶子节点也存储数据，所以查询效率不平均，并且占用的空间更大



聚簇索引和非聚簇索引

分类	含义	特点
聚簇索引	也叫 主键索引 或 聚集索引 ，叶子节点保存了 整行数据	必须有且只有一个
二级索引	也叫 非聚簇索引 ，叶子节点只保存了 索引列的数据和ID	可以有多个



回表查询和覆盖索引

！ 由上图我们可知二级索引只有**索引列**和**ID**，索引如果想要查询其他字段如gender，就必须去聚簇索引再查询一遍，这就是**回表**

具体回答：回表查询是在数据库中通过非主键索引查找数据时，先定位到索引条目，再根据索引条目中的主键或其他唯一标识符去主表中检索完整记录的过程。

如下图

所以很明显能看出来如果回表必然是比不回表速度要更慢

如果我们的sql语句是

```
select id,name from user where name = 'Arm';
```

那么就不需要再去回表，所以这也是我们平时尽量不写select *的其中一个原因

或者如果sql语句是

```
select id,name from user where name = 'Arm' and gender = 1;
```

同样会发生回表，索引这时候可以建立name和gender的联合索引

覆盖索引（有时候也叫索引覆盖）：覆盖索引是指一个索引不仅包含查找条件的列，还包含需要检索的列，使得查询可以直接使用索引本身而**无需回表**查询主数据表。

回表查询

```
select * from user where name = 'Arm';
```

回表查询

聚集索引

二级索引

联合索引和最左匹配原则

！ **联合索引**顾名思义就是多个字段组成的索引

注意：联合索引是有**顺序**的概念的，结合下图的索引是name、age、position的顺序
所以树的排序是优先按name来排序

所以自然就会有一个问题，例如：

```
select * from table where age = 28
```

这条sql语句就无法利用这个索引，因为age字段并不排第一个，所以在根节点你会发现age是30-28-30并不是有序

但是如果是name固定则age就是有序的，例如下图中叶子节点的Bill相同，则age就是30-31-32

```
select * from table where name = 'bill' and age = 28
```

这个sql语句就可以利用索引并且效率比name的单列索引更高

最左匹配原则：

通过上面的案例其实已经解释了最左匹配原则，就是我们的查询条件，必须按照联合索引的顺序来写，否则就无法走这个索引

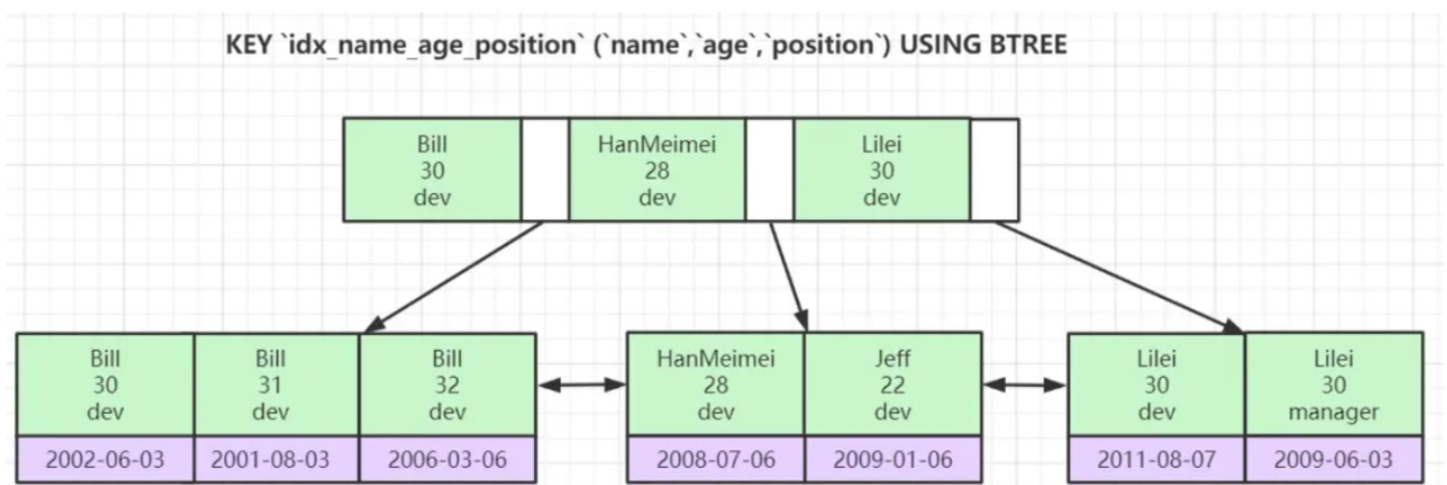
案例：

```
select * from table where age = 28 （不走）
```

```
select * from table where name = 'bill' and age = 28 （走）
```

```
select * from table where age = 28 and name = 'bill' （走）
```

特别注意的是，最左匹配sql语句里**查询条件的顺序**无关，mysql会自动优化你写的sql语句，即使条件中name在后面age在前面，优化器依然会先利用name条件



索引失效

- ❗ 1. 违反最左前缀法则
- 2. 不要在索引列上进行运算操作，索引将失效
- 3. 字符串不加单引号，造成索引失效。(类型转换)
- 4. 以%开头的Like模糊查询，索引失效
- 5. 使用!=或者<>查询
- 6. 如果某值占比过大（如 `status = 'active'` 占 90%），走索引反而比全表扫描更慢（需回表多次）。
- 7. 使用or查询而条件中有一个列没有索引（如where a = ? or b = ?，这里a和b都必须有独立的索引，才能使用索引合并，否则其他情况都会全表扫描）

总的来说大家结合那个树的结构来记忆就很好理解了

怎么判断索引是否失效

- ❗ 使用**explain**关键字查看执行计划

其中结合**key**和**key_len**判断是否命中索引

- `possible_key` 当前sql可能会使用到的索引
- `key` 当前sql实际命中的索引
- `key_len` 索引占用的大小
- `Extra` 额外的优化建议
- `type` sql连接类型

type 这条sql的连接的类型，性能由好到差为 `NULL`、`system`、`const`、`eq_ref`、`ref`、`range`、`index`、`all`

- `system`：查询系统中的表
- `const`：根据主键查询
- `eq_ref`：主键索引查询或唯一索引查询
- `ref`：索引查询
- `range`：范围查询
- `index`：索引树扫描
- `all`：全盘扫描

extra也是一个非常关键的指标，这里介绍一些常见的，注意，这些是可能同时出现的：

- `Using where`：全表扫描使用where条件过滤、**或者**使用索引访问数据，但是where中有除了该索引包含字段之外的条件
- `Using index`：使用了**覆盖索引，无需回表**
- `Using index condition`：出现了**索引下推**的情况

创建索引的原则

- ❗ 1. 数据量较大，且查询比较频繁的表 (★)
- 2. 常作为查询条件、排序、分组的字段 (★)
- 3. 字段内容区分度高 (★)
- 4. 内容较长，使用前缀索引
- 5. 尽量联合索引 (★)
- 6. 要控制索引的数量
- 7. 如果索引列不能存储NULL值，请在创建表时使用NOT NULL约束它

什么是索引下推

- ❗ **索引下推**(Index Condition Pushdown，简称ICP)，是MySQL5.6版本的新特性，它能**减少回表查询次数，提高查询效率**。

举例说明：

假设现在我们有张表 `table`，里面多个字段，其中对a，b字段建立联合索引

然后执行sql语句：`select * from table where a like '张%' and b = 1`

如果没有索引下推的情况下，由于ab字段有联合索引，但是又由于a字段的查询是范围查询所以数据库会将所有张开头的的数据查询出来后，再挨个回表查询判断是否b=1

有了索引下推后，数据库就变聪明了，他会在存储引擎层直接判断b=1这个条件，然后根据这些条件筛选出来的结果再去回表，减少回表次数

简单地说就是查询引擎将过滤条件应用到索引扫描过程中，而不是在检索出数据后再进行过滤。这样可以减少数据的访问量和I/O操作，从而提高查询性能。

这里介绍的比较简单，害怕太多文字给大家绕晕，内部其实没有这么简单，具体还涉及到了Mysql Server层和引擎层之间的交互，这里就不详细解释了，建议同学们自行网上搜索

2. Sql优化（难度★★★）

总体回答

! sql优化本身是个非常复杂的问题，要从多角度多维度来分析，不是简单的一两句话就能说清楚的

不过我们可以先总体上回答一下，看看面试官想考察哪方面的内容

答：我们会从几方面考虑，比如建表的时候，索引的创建，sql语句的编写，主从复制读写分离，量如果太大还可以采取分库分表的策略

建表

! 主要参考阿里开发手册，比如定义**字段选择合适的类型**，是否要建立**冗余字段**

参考回答：我在建表的时候主要先根据需求文档，把主流程跑一遍

然后分析里面出现的有哪些实体，然后再分析这些实体的关系

关系定了之后在考虑每个表的字段的类型，然后再考虑需不需要做冗余字段方便查询，主要是结合业务场景。最后就是建表然后写代码测试流程

创建索引

- !
1. 数据量较大，且查询比较频繁的表（★）
 2. 常作为查询条件、排序、分组的字段（★）
 3. 字段内容区分度高（★）
 4. 内容较长，使用前缀索引
 5. 尽量联合索引（★）

6. 要控制索引的数量

7. 如果索引列不能存储NULL值，请在创建表时使用NOT NULL约束它

sql语句优化

! 如果是针对sql语句的优化：

首先是**定位慢查询**，可以开启 `mysql` 的慢日志，也可以使用一些监控工具如**APM**、**Skywalking**等

定位到慢查询之后，使用 `explain` 关键字查看sql语句的执行计划，根据执行计划来判断是否命中索引，是否有回表等然后具体优化，**所以最核心的还是先判断是否能走索引**

其余就是一些平时写sql的注意事项

避免使用 `select *`

避免**索引失效**（具体如何避免参考上面内容）

避免**回表**

使用 `join` 替代 `in` 子查询

尽量用 `union all` 替代 `union`

尽量用**内连接**替代**外连接**

尽量使用 `union all` 替代 `or`

如果必须使用外连接尽量**小表驱动大表**

深度分页问题优化：下面解释

in和exists的区别

 执行逻辑上 `in` 是**值列表匹配**，`exists` 是**存在性判断**

子查询的结果越大，`in` 的性能越差，而 `exists` 越高效

因为 `in` 是先查询子查询，再将**主查询的字段逐一和子查询列表匹配**。而 `exists` 则是**取主查询的每条记录带入到子查询中判断是否能返回**

在索引的依赖上，`exists` 依赖子查询的关联索引，`in` 则依赖主表的匹配字段索引，案例如下

代码块

```
1  -- 高效: t2.id 建索引 (EXISTS 子查询关联索引)
```



```

2  SELECT * FROM t1 WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.id = t1.id AND
   t2.status=1);
3
4  -- 低效: t2.id 无索引 (子查询全表扫描, 每条主查询记录都要扫 t2)
5  SELECT * FROM t1 WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.name = t1.name AND
   t2.status=1); -- name 无索引
6
7  -- 高效: t1.id 建索引 (IN 主查询字段索引)
8  SELECT * FROM t1 WHERE t1.id IN (SELECT id FROM t2 WHERE t2.status=1); --
   t1.id 有索引
9
10 -- 低效: t1.id 无索引 (主表全表扫描, 逐一匹配 IN 列表)
11 SELECT * FROM t1 WHERE t1.name IN (SELECT name FROM t2 WHERE t2.status=1); --
   t1.name 无索引

```

深度分页问题

! 案例sql: `select * from table limit 999999,10`

深度分页太慢原因: 由于分页是按顺序从第一页开始扫描, 所以到后面分页页码太大, 就会扫描越多的数据

解决思路:

- 使用主键分页 `select * from table where id > ? order by ? limit 10`, 这种解决方案需要查询按照主键排序并且需要前端把上一页的最后一个ID传过来, 通常用于滑动式分页的页面
- 覆盖索引配合子查询:
- `select * from table as t ( select id from table order by id limit 999999,10) a where t.id=a.id`
- 由于子查询里面是覆盖索引所以会更快, 或者优化成id>()的形式, 具体大家可自行搜索
- 使用缓存, 通常是 `redis`
- 避免页码太多, 从业务解决, 比如就只能让他查看前50页

主从复制读写分离

! 由于我们的业务大部分都是读操作, 少部分是写操作, 所以

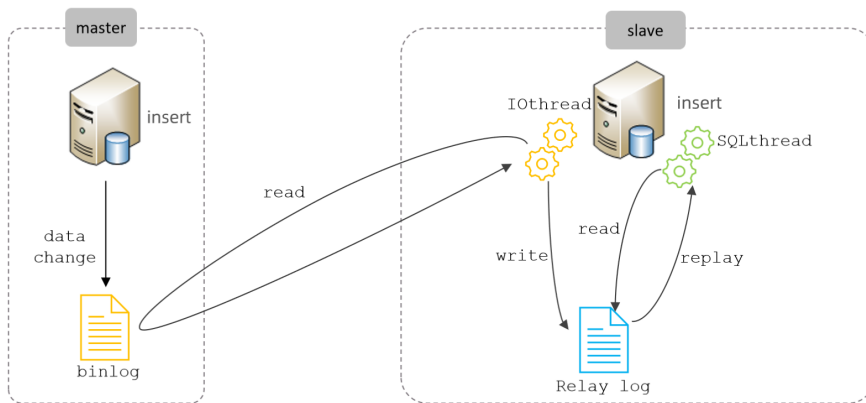
主从复制简单来说就是有一个**主节点**和**多个从节点**

从节点负责把**主节点**的数据**复制**到自己的节点中

这样我们就可以读操作去**读从节点**

写操作去**写主节点**，具体流程如下图，其中核心就是**binlog**二进制日志

注意由于从节点的数据是从主节点复制过来，所以是有一定延迟的



复制分成三步：

1. Master 主库在事务提交时，会把数据变更记录在二进制日志文件 Binlog 中。
2. 从库读取主库的二进制日志文件 Binlog，写入到从库的中继日志 Relay Log。
3. slave重做中继日志中的事件，将改变反映它自己的数据。

分区

！ 分区：

分区是 `mysql` 提供的一个功能，主要是从**物理上把数据库表文件分成多个**，只能水平拆分
具体sql语句看下面案例

其中 `partition BY RANGE` 部分就是分区的内容，具体可以按照 `RANGE`、`LIST`、`HASH`、`KEY` 来分区，具体含义大家自行搜索

优势：`mysql`提供所以我们的sql语句无需改变

缺点：要配合合理的分区方式且必须按照它提供的几种方式来分，如果分区不合理，比如查询语句横跨好几个分区，则效率反而会下降

代码块

```
1 CREATE TABLE employees(  
2     id int not null,  
3     store_id int not null  
4 )partition BY RANGE (store_id)(  
5     partition p0 values LESS THAN (6),  
6     partition p0 values LESS THAN (11),  
7     partition p0 values LESS THAN (16),  
8     partition p0 values LESS THAN (21),  
9 )
```

分表

!

分表是我们程序员自己的行为，对数据库来说就是多张表

比如我们可以把订单表分成 `order_202408`、`order_202409`、`order_202410` 依次类推，每个月一张表，这是水平拆分

也可以把宽表分成多张表，比如原来是user表，表里50多个字段，拆成 `user` 和 `user_detail`，其中 `user` 只保留常用的10个字段，其余不常用的字段都在 `user_detail` 中

优势：对比分区更灵活，想怎么分都可以

缺点：表名变了，所以我们原来的sql语句也得跟着变，比如之前的sql是

```
select * from order where create_time > '2024-08-01';
```

分表之后可能就要变成

```
select * from order_202408 where create_time > '2024-08-01';
```

这里我们可以使用MyBaitsPlus提供的动态表名插件

同时如果设计不合理，跨表查询不方便

分库

!

分库也分为**水平拆分**和**垂直拆分**

水平拆分就比如通过hash的方式把数据存储到不同的库，比如 `user_1`、`user_2` 等

垂直拆分就比如我们根据业务拆分成不同的库，比如原本只有一个库，拆成 `user`、`order` 等

分库也会带来新的问题比如：分布式事务，跨节点查询等

而在框架上目前有 `sharding-sphere` 和 `mycat` 比较常用与分库，同学们自行了解

3. 事务相关(难度★★★★★)

事务隔离级别

隔离级别	脏读	不可重复读	幻读
读未提交Read uncommitted	√	√	√
读已提交Read Committed	×	√	√

可重复读（默认） Repeatable Read	×	×	√
串行化Serializable	×	×	×

！ 隔离级别顾名思义就是针对事务的四大特性中的隔离性

所谓隔离其实指的一个事务和另一个事务之间能否做到数据隔离

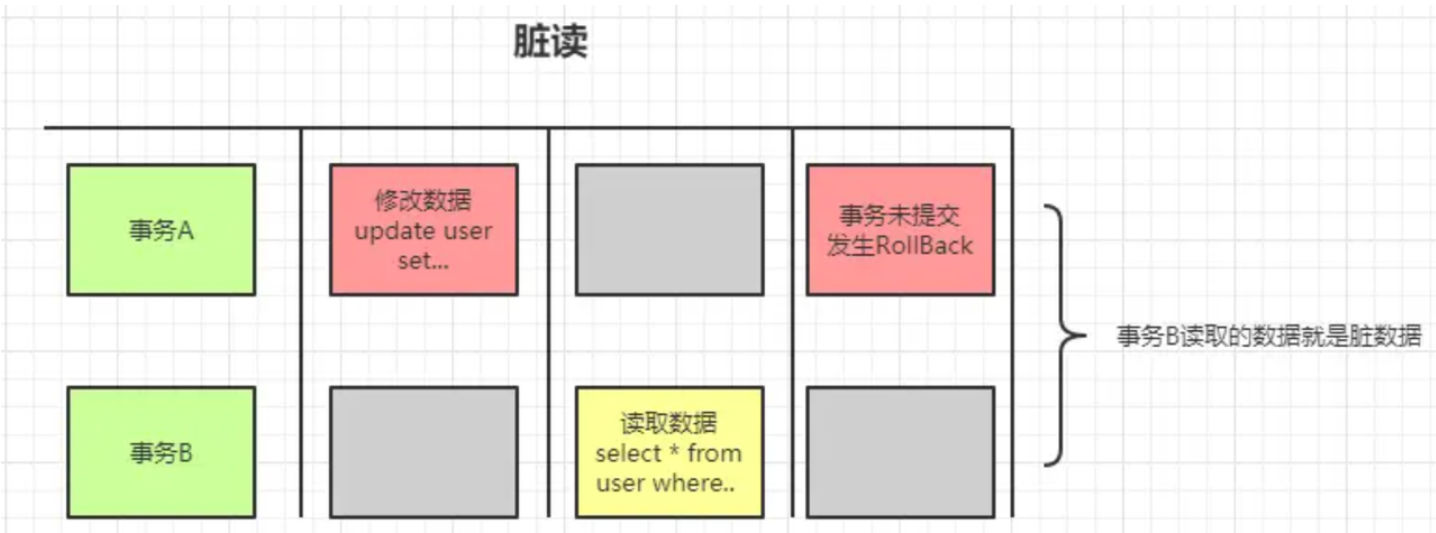
而表格中由上往下隔离级别依次越来越高，出现脏读、不可重复读、幻读的问题也越来越少，同时性能也越来越差

注意：mysql的可重复读级别并不是无法解决幻读，而是某些情况下的幻读无法解决，具体可以采取mysql的mvcc和锁来解决幻读

脏读、不可重复读、幻读

！ 脏读

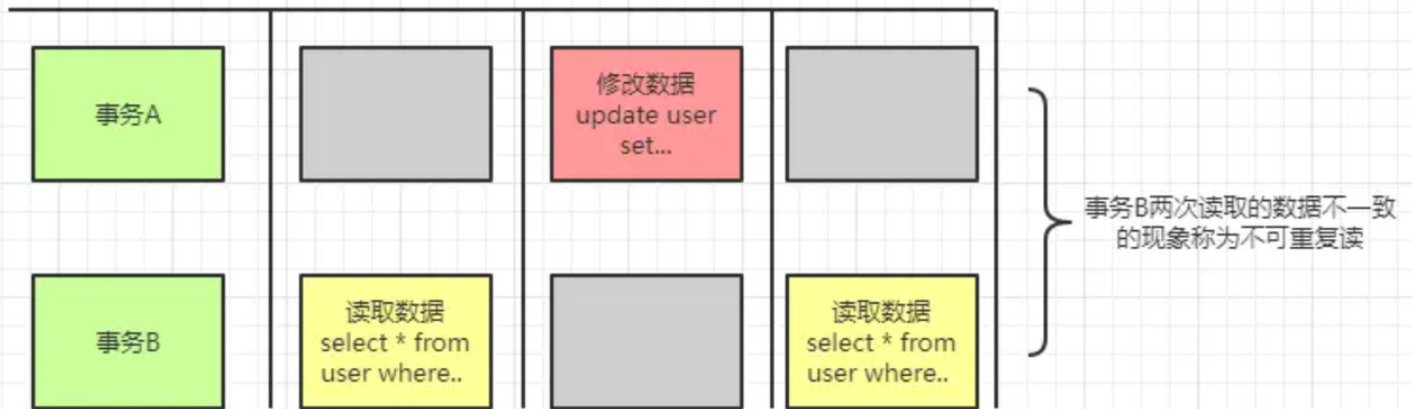
事务B读取到了事务A未提交的数据



！ 不可重复读

事务B读取两次数据，由于事务A修改了这条数据并且提交了事务导致事务B两次读取的结果不一样

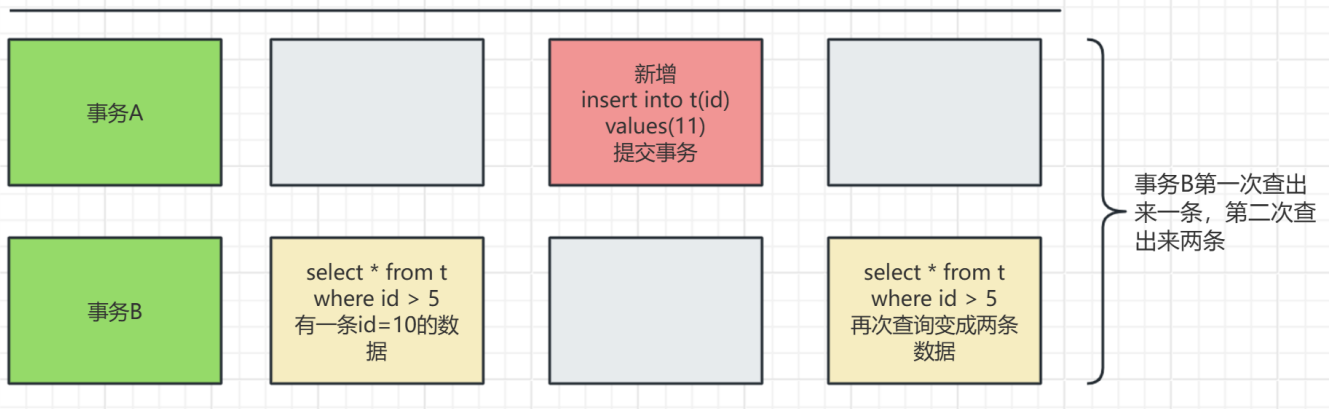
不可重复读



! 幻读

幻读（Phantom Read）是数据库事务并发执行时可能出现的一种问题，它发生在**同一个事务内**，先后两次执行**相同范围的查询**，但第二次查询返回的结果集却包含了第一次查询中未出现的行（即“幻影”行）。

幻读



! 其中这三种最容易搞混的就是幻读和不可重复读

幻读强调的是**集合或者范围内的增减**，而不是单条数据的更新

简单来说：

- 不可重复读是**行的值变了**，解决**读同一行不一致**的问题；
- 幻读是**行的数变了**，解决**读结果集行数不一致**的问题。

MySQL InnoDB 的特殊说明：

值得注意的是，标准的SQL规范中，默认**可重复读级别是无法避免幻读的**。但是，MySQL 的 InnoDB 存储引擎通过一种叫做**临键锁Next-Key Locking（行锁+间隙锁）**的机制，在**可重**

复读级别下，很大程度上避免了幻读的发生。它会在范围查询时锁定这个范围，阻止其他事务在这个范围内插入新的记录。

Mysql三大日志

! `binlog` 二进制日志，所以里面记录的是数据库中的**DDL、DML**操作，用**二进制**存储，可以通过 `mysqlbinlog` 工具解析 `binlog` 二进制文件，输出 SQL，注意使用与 MySQL 相同版本的客户端解析。

开启 `binlog`

在 `mysqld` 配置文件中加上，参数为 `binlog` 文件名前缀。如下配置

```
[mysqld]
log-bin                        = mysql-bin.log
gtid_mode                     = ON
enforce-gtid-consistency      = ON
```

常用于**主从同步**以及 `Canal` 的 `mysql` 和 `ElasticSearch` 或者 `Redis` 的数据同步

! `undo log` 叫做**回滚日志**也叫**撤销日志**

在事务执行变更操作之前需要先将相反的操作写入 `undo log`，通过它可以让事务回滚操作

`undo log` 也是实现多版本控制（`MVCC`）的基础。

`redo log`：记录的是数据页的物理变化，服务宕机可用来同步数据

`redo log` 叫重做日志，它**保证了事务的持久性**，而 `undo log` **保证了事务的原子性和一致性**。

具体过程不详细解释了，参考面试专题

! `InnoDB`存储引擎是**以页为单位**来管理存储空间的。在真正访问页面之前，需要把在**磁盘上的页缓存到内存中的** `Buffer Pool` 之后才可以访问。所有的变更都必须先更新**缓冲池**中的数据，然后缓冲池中的**脏页**会以一定的频率被刷入磁盘（`Check Point`机制），通过缓冲池来优化CPU和磁盘之间的鸿沟，这样就可以保证整体的性能不会下降太快。

具体流程参考面试专题

多版本并发控制MVCC

！关于MVCC的详细流程，这里不展开详细介绍，大家参考面试专题或者B站视频，一定要**先去看懂大概流程**再回头来看下面的回答，就会好记很多

我这里只说重点：

首先MVCC的意思是**多版本并发控制**，指的是一条数据的多个版本，并使其读写操作没有冲突，只在**RC（读已提交）**和**RR（可重复读）**级别下工作

底层实现主要分为**三大块**：**隐藏字段**，**undo log**，**readView读视图**

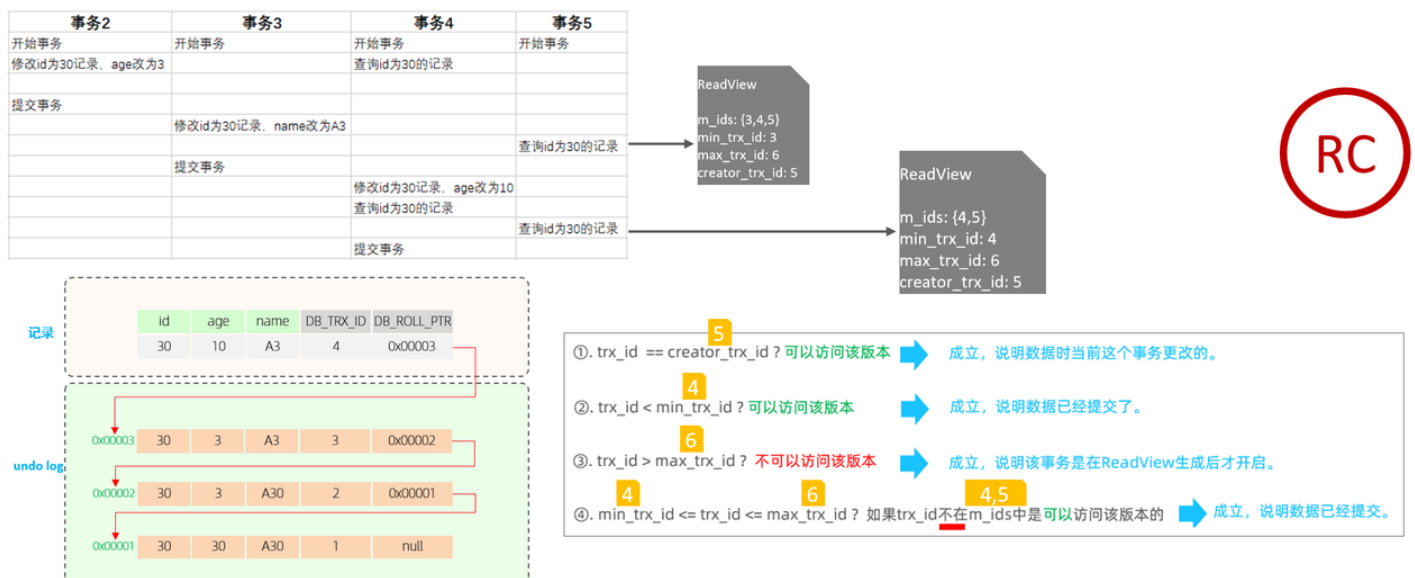
隐藏字段指的是mysql给每张表都设置了几个隐藏字段，有 **trx_id事务id** 记录事务的id并且自增，**roll_pointer回滚指针** 记录不同事务修改数据的版本，并且通过该指针形成一个链表

undo log 作用是记录回滚日志，存储老版本的数据，内部就会形成一个版本链，在多个事务并行操作某一行，记录不同事务修改数据的版本，通过 **roll_pointer** 形成链表

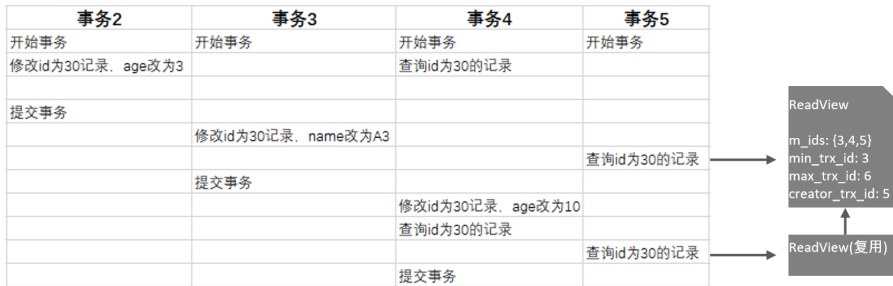
ReadView读视图 解决一个事务到底查询那个版本的问题，mysql内部定义了一些匹配规则用于当前事务id判断选择什么版本，不同隔离级别的快照读是不一样的，最终访问的版本也不一样

如果是 **RC读已提交** 级别，则**每次执行快照读**都生成 **ReadView读视图**

如果是 **RR可重复读** 级别，则在事务**第一次执行快照读**的时候生成 **ReadView读视图**，后续读操作都用这个 **ReadView视图**



RR隔离级别下，仅在事务中第一次执行快照读时生成ReadView，后续复用该ReadView。



Mysql中的锁

！ 这里说的锁都是Mysql本身提供的，而不是我们代码中所使用的，由于课程中没有讲过该内容，平时日常开发也很少使用，所以内容很多不利于理解，这里建议同学们少食多餐，每次学习一点点

也不是悲观锁乐观锁这种概念，而是具体的锁

Mysql的InnoDB存储引擎按锁颗粒度来分可以分成

行级锁和表级锁，顾名思义行锁就是锁具体某行数据，表锁就是锁整张表

具体的锁又分为**共享锁(S)**，**排他锁(X)**，**意向共享锁(IS)**，**意向独占锁(IX)**，**间隙锁**，**Next-Key临键锁**等等

！ 共享锁Shared Locks(S锁)

加上S锁的记录，允许其他事务再次上S锁，但是不允许其他事务上X锁

行级S锁具体语句：`select .... Lock in share mode`

表级S锁具体语句：`select` 隐式上锁，`lock table tableName read` 手动显式上锁，但是要使用 `unlock table tableName` 释放锁

！ 排他锁Exclusive Locks(X锁)

加了X锁的记录，不允许其他事务再加S锁和X锁

行级X锁具体语句：`select ... for update`

表级X锁具体语句：`insert、update、delete` 语句会隐式上锁，`lock table tableName write` 手动显式上锁，同样也要手动释放锁

！ 意向锁Intention Locks

意向锁的存在是为了协调行锁和表锁的关系，支持多颗粒度的锁共存，当一个事务需要对某个资源进行写操作时，会先尝试获取其意向锁，如果此时已经被其他事务占用了意向锁或排他锁，则该事务则需要等待

例子：事务A修改 `user` 表的记录 `R`，会给记录 `R` 上一个行级的**排他锁(X)**

同时也会给 `user` 表上一把**意向排他锁(IX)**

如果这时事务B也要给 `user` 表上一个表级的**排他锁**就会被阻塞

Q1：为什么意向锁是表级锁

因为当我们要加X锁时，需要根据意向锁来判断这个表里有没有数据行被锁定

如果意向锁是行锁，那就需要遍历每一行数据

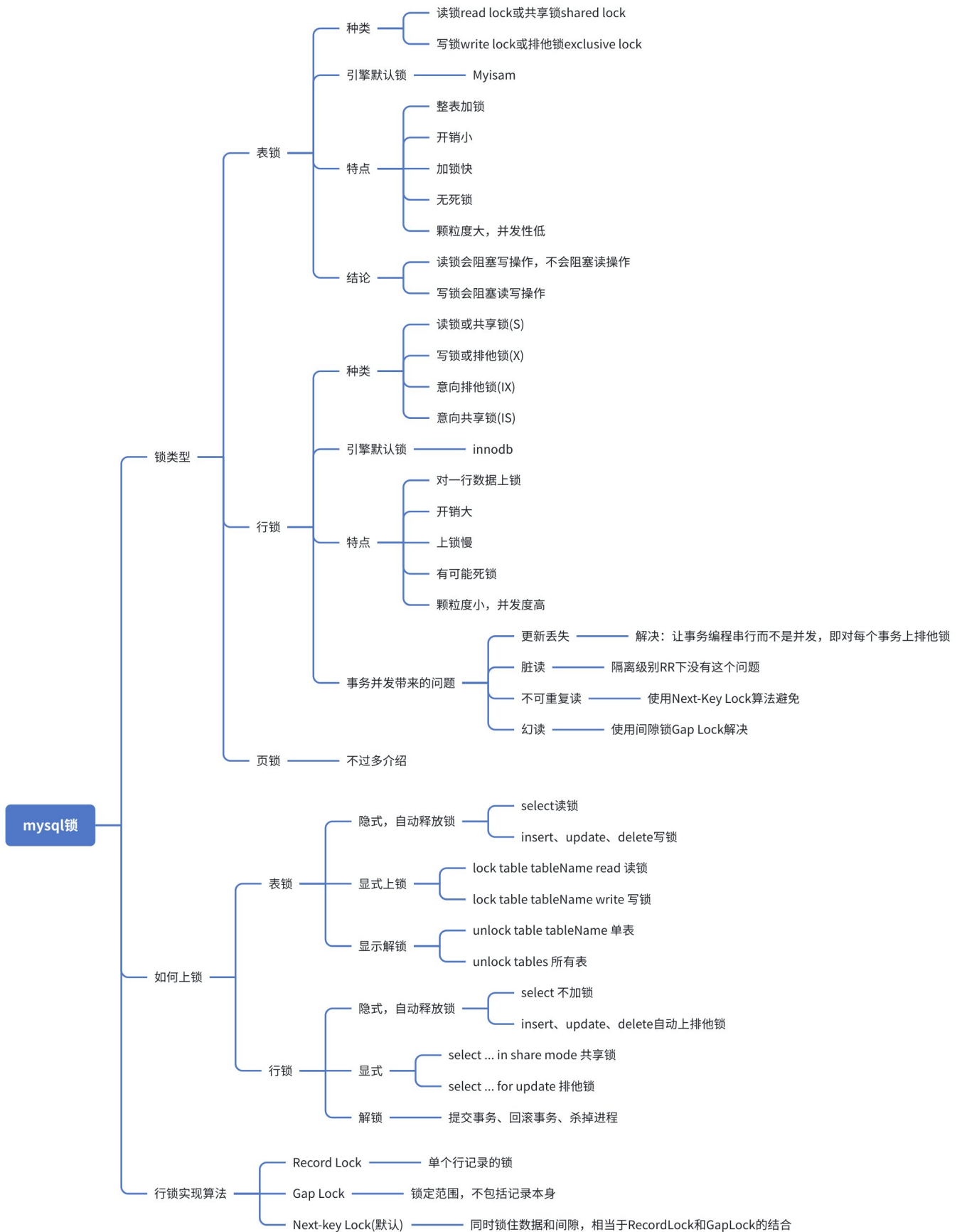
如果意向锁是表锁，那就直接确认即可

Q2：意向锁怎么支持表锁和行锁共存

这里共存指的是数据库支持表锁、行锁同时存在，而不是所有情况都支持一个表里既有事务A持有行锁，又有事务B持有表锁，因为表一旦被上了表级写锁，那就不能再上一个行级锁

如果事务A对某一行记录上锁，那么其他事务就不能修改这一行数据，这就和 “事务B锁住整个表就能修改表的任意一行” 形成冲突，所以如果没有意向锁，想要让表锁和行锁共存会有很多问题，于是正和Q1里提到的答案一样，数据库就不需要判断每一行是否都有锁，只需要判断意向锁是否存在即可。

是否兼容	事务A上了IS	IX	S	X
事务B能否上IS	可以	可以	可以	不行
IX	可以	可以	不行	
S	可以	不行	可以	不行
X	不行	不行	不行	不行



! 锁的选择

1. 如果更新条件没有走索引，例如执行 `update test set name='hello' where name='world';`，此时会进行全表扫描，扫表的时候，要阻止其他任何的更新操作，所以上升为表锁。
2. 如果更新条件为索引字段，但是并非唯一索引（包括主键索引），例如执行 `update test set name='hello' where code=9` 那么此时更新会使用 `Next-Key Lock`。

使用 `Next-Key Lock` 的原因：

- 首先要保证在符合条件的记录上加上排他锁，会锁定当前非唯一索引和对应的主键索引的值；
- 还要保证锁定的区间不能插入新的数据。
- 如果更新条件为唯一索引，则使用 `Record Lock`（记录锁）

! 查看数据库中锁的使用情况

`SHOW PROCESSLIST` 命令中可以通过state列，查询事务是否获取了锁。可能的锁状态包括：

- Locked
- Waiting for lock
- Lock wait timeout exceeded

还可以通过 `INFORMATION_SCHEMA`（mysql自带数据库）中的 `INNODB_LOCKS` 和 `INNODB_LOCK_WAITS` 两张表中查看

4. 其他相关（难度★）

Mysql和Oracle的区别

! Mysql 和 Oracle 都是关系型数据库

Mysql社区版是开源免费的，Oracle是商用数据库需要收费

性能上来说Oracle更强，功能上Oracle也更多

对于我们程序员来说比较明显的区别在于Sql的使用上

1. 分页

Mysql分页使用 `limit`，**Oracle**分页使用 `rownum`

2. 空串

Mysql中 `''` 空串 和 `null` 是不同的，Oracle中没有 `''` 和 `null` 是一样的

3. 字段

字段类型上数值类型Mysql分为 `int` `double` 等，Oracle用 `number` 类型

4. 自增

自增ID上Mysql使用 `auto_increment` 自增，Oracle使用**序列**

Mysql和PostgreSql的区别

! PostgreSQL也叫PGsql

总体上来说大部分场景下PGSQL的性能更好，数据类型也更丰富，用法也类似Mysql，所以目前越来越多的国内企业选择使用PG。同学们务必要了解这个东西
具体对比区别这里不再一一列举，同学们自行查询

存储过程

! **存储过程，就像是一个预先准备好的小工具包或者提前写好的代码。**举例来说我有一些经常要做的复杂任务，比如要从好几个表里面查数据，然后做一些计算，再把结果存到另一个地方。如果每次都写一大串 SQL 语句，这样比较麻烦。存储过程就是把这些复杂的操作打包起来，你只需要给它一些参数，它就帮你把事情办好。

存储过程有几个好处。

首先，它能让你的代码更简洁。不用每次都重复写那些复杂的 SQL 语句，直接调用存储过程就行。

其次呢，它可以提高效率。因为存储过程是在数据库服务器上预先编译好的，执行起来比每次临时写 SQL 语句要快得多。还有哦，如果你的数据库有很多人用，你可以只让他们调用存储过程，而不让他们直接操作底层的表，这样更安全。而且，一旦你写好了一个存储过程，在不同的项目里都可以用，很方便呢。

比如说，你有一个电商网站，要经常计算某个用户的订单总额。你就可以写一个存储过程，接收用户 ID 作为参数，然后去查订单表，把这个用户的所有订单金额加起来，返回结果。以后每次要算这个用户的订单总额，直接调用这个存储过程就好啦。

以下是存储过程的写法

```

1 DELIMITER //
2 -- 创建名为 CalculateUserOrderTotal 的存储过程，用于计算指定用户的订单总额
3 CREATE PROCEDURE CalculateUserOrderTotal (IN in_user_id INT, OUT
  out_total_amount DECIMAL(10,2))
4 BEGIN
5     -- 从 orders 表中，根据输入的用户 ID，计算订单金额总和，并存储到
    out_total_amount 变量中
6     SELECT SUM(order_amount) INTO out_total_amount
7     FROM orders
8     WHERE user_id = in_user_id;
9     END //
10 DELIMITER ;

```

以下是调用存储过程的写法

代码块

```

1 -- 设置一个用户 ID 变量并赋值
2 SET @user_id = 123;
3 -- 设置用于存储订单总额的变量并初始化为 0
4 SET @total = 0;
5
6 -- 调用存储过程 CalculateUserOrderTotal，传入用户 ID 和用于接收结果的变量
7 CALL CalculateUserOrderTotal(@user_id, @total);
8
9 -- 选择存储过程计算出的订单总额变量进行查看
10 SELECT @total AS user_order_total;

```

其余还有一些比较简单的，比如Mysql中的存储引擎，事务的四大特性，Mysql的常用函数，Mysql的字段类型，还有一些查询手写sql语句的，比如去掉一个表中的重复数据，多对多关系连表查询等，这些这里不再赘述属于同学们的基本功

5.场景题

5s内100w数据怎么快速插入另一张表

类似的问题还有怎么快速插入1000万数据等

！ 使用 `LOAD DATA INFILE` 工具，将数据整理成csv格式文件，通过加载文件的方式插入

代码块

```
1  -- 假设有一个CSV文件data.csv, 包含id, name, age三列
2  LOAD DATA INFILE '/path/to/data.csv' INTO TABLE users
3  FIELDS TERMINATED BY ',' ENCLOSED BY '"'
4  LINES TERMINATED BY '\n'
5  IGNORE 1 ROWS;
```

！ 使用代码的话，一般使用JDBC批处理，批量插入，但是要注意一些配置调整
如调整 `innodb_log_file_size` 和 `innodb_buffer_pool_size` 等参数
临时禁用索引，手动处理事务
在数据连接开启 `rewriteBatchedStatements = true`

你一般是怎么设计表的

！ 首先要分析具体业务，先把整个业务的**流程**给捋顺，包括数据流转和状态流转
再根据业务流程分析出里面的**业务实体**，也就是大概有那些表
以及这些表之间的关系，然后就看具体的一对多、多对多关系来设计表结构，选择合适的字段

你们系统数据量有多大

！ 这个看你们项目具体情况，一般来说海量数据一般出现在流水表（订单记录，积分记录，点赞记录）等，随着时间的推移数据量会越来越大，根据自己系统的用户量估一个数。

然后可能追问你采取什么策略
下一个问题可能会问海量数据的存储策略，这里面就考虑**分区、分库、分表、集群**甚至直接换其他数据库比如 `TiDB`

数据库中的死锁怎么排查

！ 一、使用 `SHOW ENGINE INNODB STATUS` 命令，在输出的死锁信息部分，可以看到涉及死锁的事务正在操作的表名称。

例如：

LATEST DETECTED DEADLOCK

*** (1) TRANSACTION:

TRANSACTION 12345, ACTIVE 0 sec, process no 1234, OS thread id 11111 starting index read

mysql tables in use 1, locked 1

LOCK WAIT 2 lock struct(s), heap size 1136, 1 row lock(s)

MySQL thread id 5678, query id 98765 UPDATE table1 SET column1 = value1 WHERE id = 10

...

*** (2) TRANSACTION:

TRANSACTION 67890, ACTIVE 0 sec, process no 1235, OS thread id 22222 starting index read, thread declared inside InnoDB 5000

mysql tables in use 1, locked 1

4 lock struct(s), heap size 1136, 2 row lock(s)

MySQL thread id 5679, query id 98766 UPDATE table1 SET column2 = value2 WHERE id = 10

...

! 二、查询 `information_schema` 数据库

1. 查询 `INNODB_LOCKS` 表:

- 可以查看当前 InnoDB 存储引擎中的**锁信息**，包括锁定的表和行。
- 示例查询: `SELECT * FROM information_schema.INNODB_LOCKS;`

2. 查询 `INNODB_TRX` 表:

- 可以查看当前正在运行的**事务信息**，包括事务正在操作的表。
- 示例查询: `SELECT * FROM information_schema.INNODB_TRX;`